

ACCELERAZIONE DI SIMULAZIONI MONTE CARLO APPLICATE AD UN ASSET FINANZIARIO

...

**Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864**

COS'È UNA SIMULAZIONE MONTECARLO?

Metodo statistico frequentemente usato in vari ambiti:

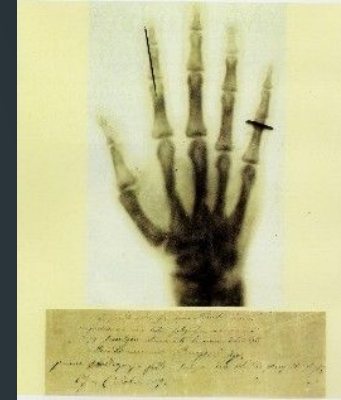
❖ *Finanza e Assicurazioni*

- *Valutazione di opzioni*
- *Gestione del portafoglio*
- *Pianificazione previdenziale*



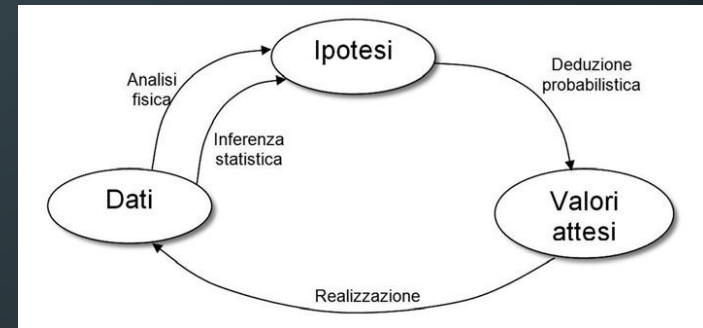
❖ *Industria*

- *Analisi della Tolleranza*
- *Affidabilità e Manutenzione*
- *Analisi dei Rischi*



❖ *Fisica e Scienza dei Materiali*

- *Fisica delle particelle*
- *Radioterapia*



IL PROBLEMA

Investendo una somma x in un asset finanziario in un istante T_0 , quale valore ci si può aspettare di ottenere dal mio investimento ad un istante T_1 ?

**Per provare a rispondere si può usare il metodo
Monte Carlo!**

DINAMICA STOCASTICA DEI PREZZI

IL MOTO BROWNIANO GEOMETRICO (GBM)

IPOTESI FONDAMENTALE

L'evoluzione del sottostante S_t segue un processo descritto dalla seguente Equazione Differenziale Stocastica (SDE):

LEGENDA

- ❑ **Z** Variabile casuale standard $N(0,1)$
- ❑ **S_t** Prezzo dell'indice al tempo t .
- ❑ **μ** Drift (tendenza media), del dataset.
- ❑ **σ** Volatilità storica annualizzata.
- ❑ **W_t** Processo di Wiener (Moto Browniano)

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

$$\text{Prezzo}_{t+\Delta t} = \text{Prezzo}_t \cdot e^{\left[\left(\mu - \frac{1}{2}\sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \right]}$$

*Usando i log rendimenti
si possono sfruttare le loro proprietà*

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

$$\ln(P_{t+1}/P_0) = \sum_{i=0}^t r_{\log,i}$$

IL METODO MONTE CARLO

Si basa sulla **Legge dei Grandi Numeri** (LLN)

Il **valore atteso** di una funzione $f(S_t)$ viene *approssimato* dalla media campionaria di N realizzazioni indipendenti:

$$\mathbb{E}[f(S_T)] \approx \frac{1}{N} \sum_{i=1}^N f(S_{T,i})$$

L'**errore standard** dello stimatore Monte Carlo *decade* secondo la proporzione:

$$\epsilon \propto \frac{1}{\sqrt{N}}$$

MONTE CARLO: SIMULAZIONI SEQUENZIALI NAIF CPU

```
// Parametri Iniziali
double driftTerm = (drift - 0.5 * volatilita * volatilita) * T_YEARS;
double volTerm = volatilita * std::sqrt(T_YEARS);

for (int i = 0; i < nSimulations; ++i) {
    double Z = distribution(generator); // Generazione numero casuale
    double ST = S0 * std::exp(driftTerm + volTerm * Z); // Formula GBM
    simulatedPrices[i] = ST;
}
```

MONTE CARLO: SIMULAZIONI ACCELERATE NAIF GPU

```
__global__ void monteCarloKernel(double* out,int n,double S0,double
driftTerm,double volTerm,unsigned long seed) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    curandStatePhilox4_32_10_t state;
    curand_init(seed, idx, 0, &state);

    double Z = curand_normal_double(&state);
    out[idx] = S0 * exp(driftTerm + volTerm * Z);
}
```

MONTE CARLO: SIMULAZIONI ACCELERATE NAIF GPU

```
double driftTerm = (drift - 0.5 * volatilita * volatilita) * T_YEARS;
double volTerm   = volatilita * std::sqrt(T_YEARS);
// Allocazione variabile su GPU per simulazioni
double* d_sim;
cudaMalloc(&d_sim, N_SIMULATIONS * sizeof(double));
// Definizione griglia e blocchi 1D e 1D
dim3 blockDim(256, 1, 1);
dim3 gridDim((N_SIMULATIONS + blockDim.x - 1) / blockDim.x, 1, 1);

monteCarloKernel<<<gridDim, blockDim>>>(d_sim, N_SIMULATIONS, S0, driftTerm, volTerm, SEED);
cudaDeviceSynchronize();
// Trasferimento prezzi simulati da GPU a CPU
std::vector<double> simulatedPrices(N_SIMULATIONS);
cudaMemcpy(simulatedPrices.data(), d_sim, N_SIMULATIONS * sizeof(double), cudaMemcpyDeviceToHost);
cudaFree(d_sim);
```

SOLUZIONE GPU (ITERAZIONI 1, 3)

```
__global__ void monteCarloKernel(float* out, int n, float
S0, float driftTerm, float volTerm, unsigned long seed) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    curandStatePhilox4_32_10_t state;
    curand_init(seed, idx, 0, &state);
    float Z = curand_normal(&state);
    out[idx] = S0 * exp(driftTerm + volTerm * Z);
}
```

L'ordinamento ha tempi rilevanti al pari della simulazione Monte Carlo

- ❖ Sostituzione tipi double con float
- ❖ Aggiunto sort parallelo con libreria CUDA thrust

```
// Wrapping del puntatore raw per thrust
thrust::device_ptr<float> dPtr(dSim);
thrust::sort(dPtr, dPtr + nSimulations);
```

SOLUZIONE GPU (ITERAZIONI 4, 5)

Semplificazione operazioni e PARALLELIZZAZIONE

```
curandStatePhilox4_32_10_t state;  
curand_init(seed, tid, 0, &state);  
  
// Genero 4 numeri casuali in un colpo solo (istruzione vettoriale)  
float4 Z = curand_normal4(&state);  
  
float4 res;  
// Funzione esponenziale approssimata  
res.x = S0 * __expf(driftTerm + volTerm * Z.x);  
res.y = S0 * __expf(driftTerm + volTerm * Z.y);  
res.z = S0 * __expf(driftTerm + volTerm * Z.z);  
res.w = S0 * __expf(driftTerm + volTerm * Z.w);  
  
// Scrittura vettorizzata in memoria globale (1 transazione per 4 float)  
reinterpret_cast<float4*>(out)[tid] = res;
```

SOLUZIONE GPU (ITERAZIONI 6, 8, 9)

Aumentate le simulazioni per thread

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
int totalThreads = gridDim.x * blockDim.x;
// Ogni thread ha il proprio stato curand
curandStatePhilox4_32_10_t state;
curand_init(seed, tid, 0, &state);
float4 Z;
float4 res;
for (int i = 0; i < N_CYCLES; i++) {
    int globalFloat4Idx = tid + (i * totalThreads);
    // Controllo bounds (moltiplicato per 4 perché n è il numero di float, non float4)
    if (globalFloat4Idx * 4 >= n) return;
    Z = curand_normal4(&state);
    res.x = S0 * __expf(driftTerm + volTerm * Z.x);
    res.y = S0 * __expf(driftTerm + volTerm * Z.y);
    res.z = S0 * __expf(driftTerm + volTerm * Z.z);
    res.w = S0 * __expf(driftTerm + volTerm * Z.w);
    reinterpret_cast<float4*>(out)[globalFloat4Idx] = res;}
```

SOLUZIONE GPU (ITERAZIONE 7)

VICOLI CIECHI TENTATI:

- ❖ Pinned Memory
- ❖ Kernel Separati per puro calcolo e allocazione risorse

A cosa serve la pinned memory se dovremmo ordinare i dati?

Lanciare due kernel sequenziali è costoso se non puoi fare latency hiding

JUST KEEP IT SIMPLE.

Come si possono alleggerire i trasferimenti di memoria?

SOLUZIONE GPU (ITERAZIONI 9, 10)

Alleggerito il trasferimento di dati *device to host*

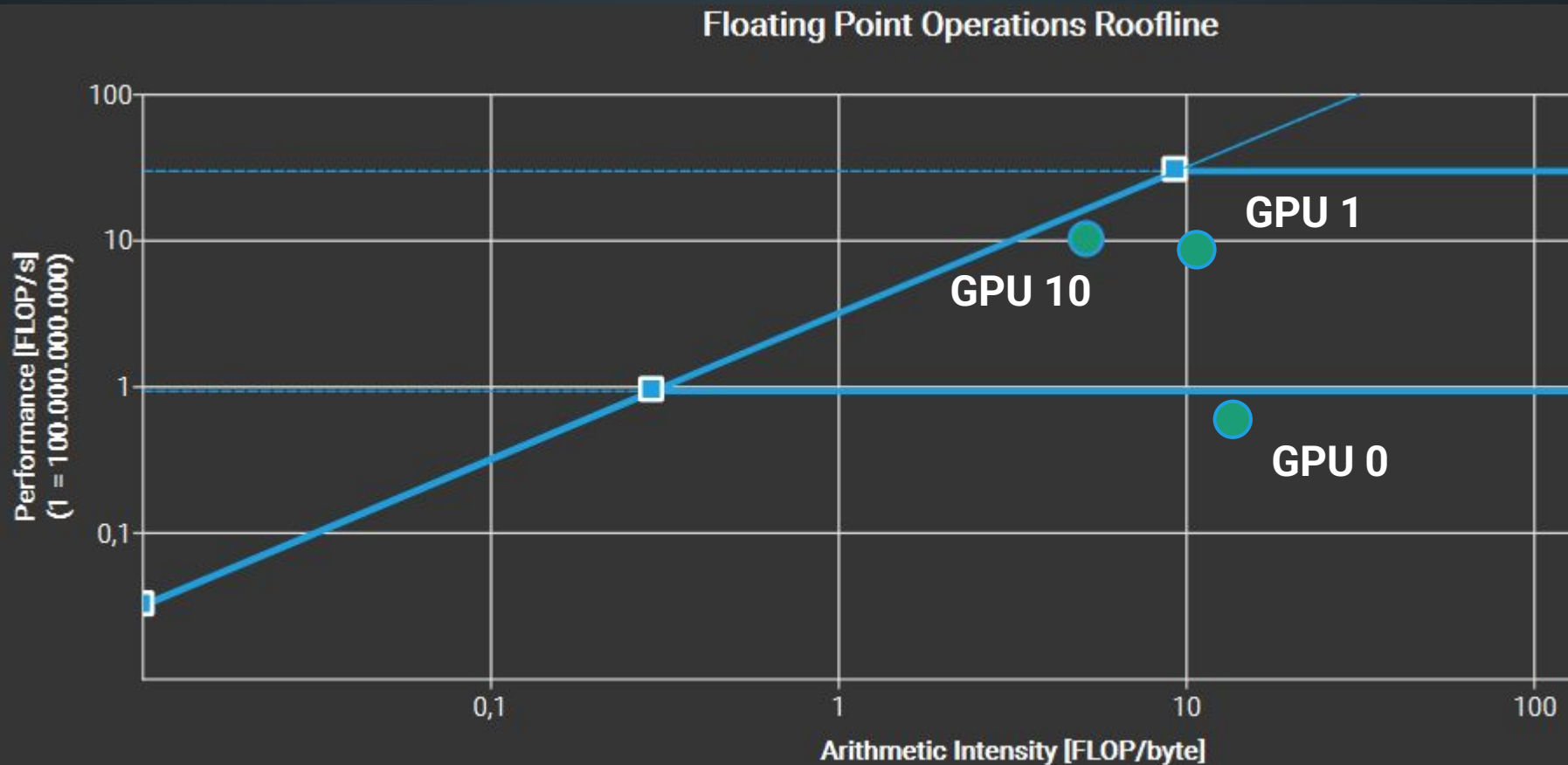
```
// dSimDevice = puntatore all'array sul device che abbiamo appena ordinato
// Analisi dei risultati
float priceWorst, priceMed, priceBest;

int idxWorst = (int)(nSimulations * 0.01f);
int idxMed    = (int)(nSimulations * 0.50f);
int idxBest   = (int)(nSimulations * 0.99f);

// Copia da (dSimDevice + offset) a variabile CPU
cudaMemcpy(&priceWorst, dSimDevice + idxWorst, sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(&priceMed,    dSimDevice + idxMed,    sizeof(float), cudaMemcpyDeviceToHost);
cudaMemcpy(&priceBest,   dSimDevice + idxBest,   sizeof(float), cudaMemcpyDeviceToHost);
```

Trasferire tutto l'array è molto più lento rispetto a solo i valori di interesse

ROOFLINE MODEL

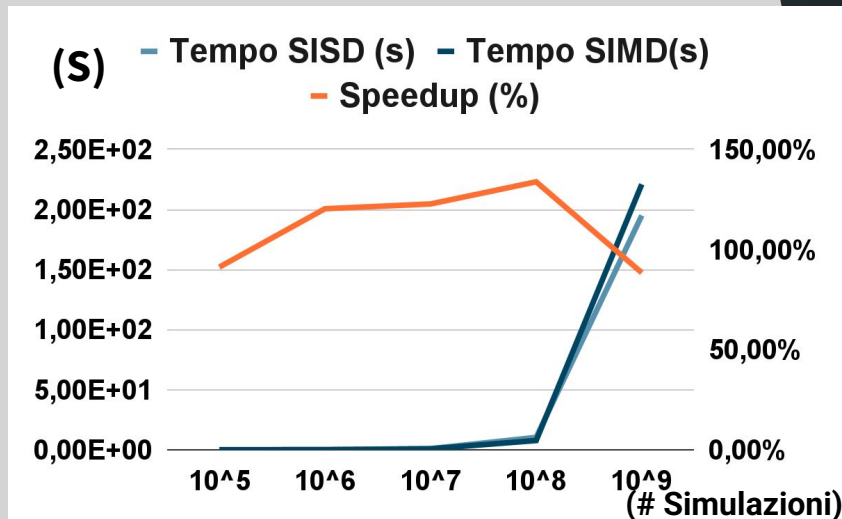


SOLUZIONE SIMD

Soluzione sequenziale naif come base e sostituzione
dell'algoritmo RNG C++ con Xorshift

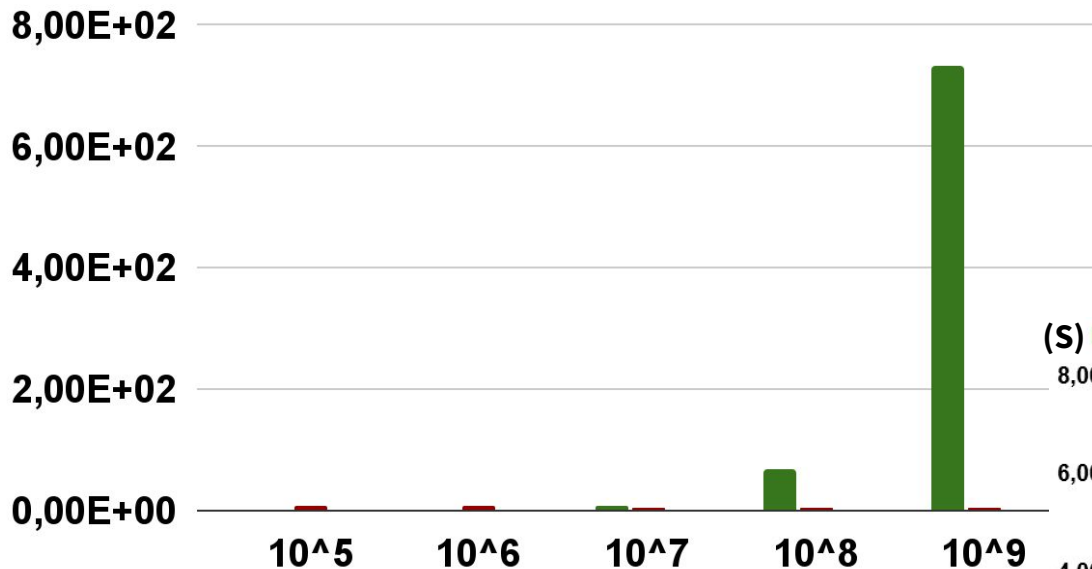
```
// Loop Principale
for(long i = 0; i + 3 < nSimulations; i += 4){
    __m128 Z = normalSIMD(rngState);
    __m128 X = _mm_add_ps(driftV, _mm_mul_ps(volV, Z));
    _mm_store_ps(out, X);
    for(int k = 0; k < 4; ++k){
        out[k] = S0 * std::exp(out[k]);
    }
    __m128 ST = _mm_load_ps(out);
    _mm_storeu_ps(&simulatedPortfolioValues[i], ST);
}

// Stato random per parte sequenziale
uint32_t seqState = 0x1A2B3C4Du + nSimulations;
// Tail sequenziale finale (da 1 a 3 simulazioni)
for(long i = (nSimulations & ~3); i < nSimulations; ++i){
    float Z = normalSeq(seqState);
    simulatedPortfolioValues[i] = S0 * std::exp(driftTerm + volTerm * Z);}
```

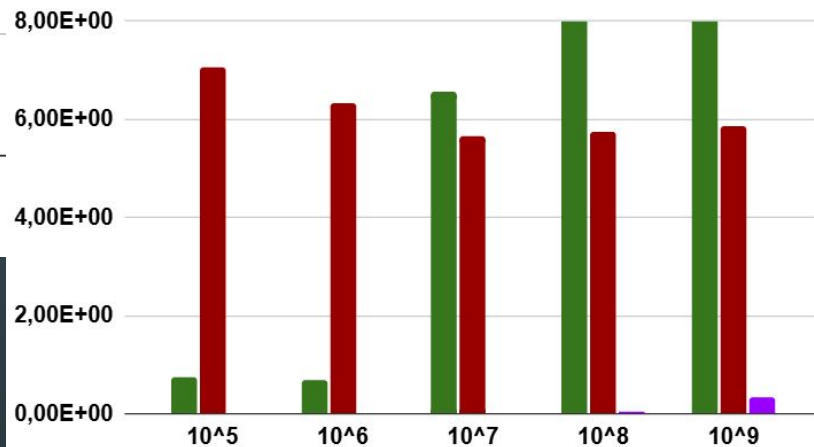


RISULTATI OTTENUTI E ANALISI

(S) ■ Tempo CPU ■ Tempo GPU Naive ■ Tempo GPU Opt



(S) ■ Tempo CPU (s) ■ Tempo GPU Naive (s) ■ Tempo GPU Opt (s)

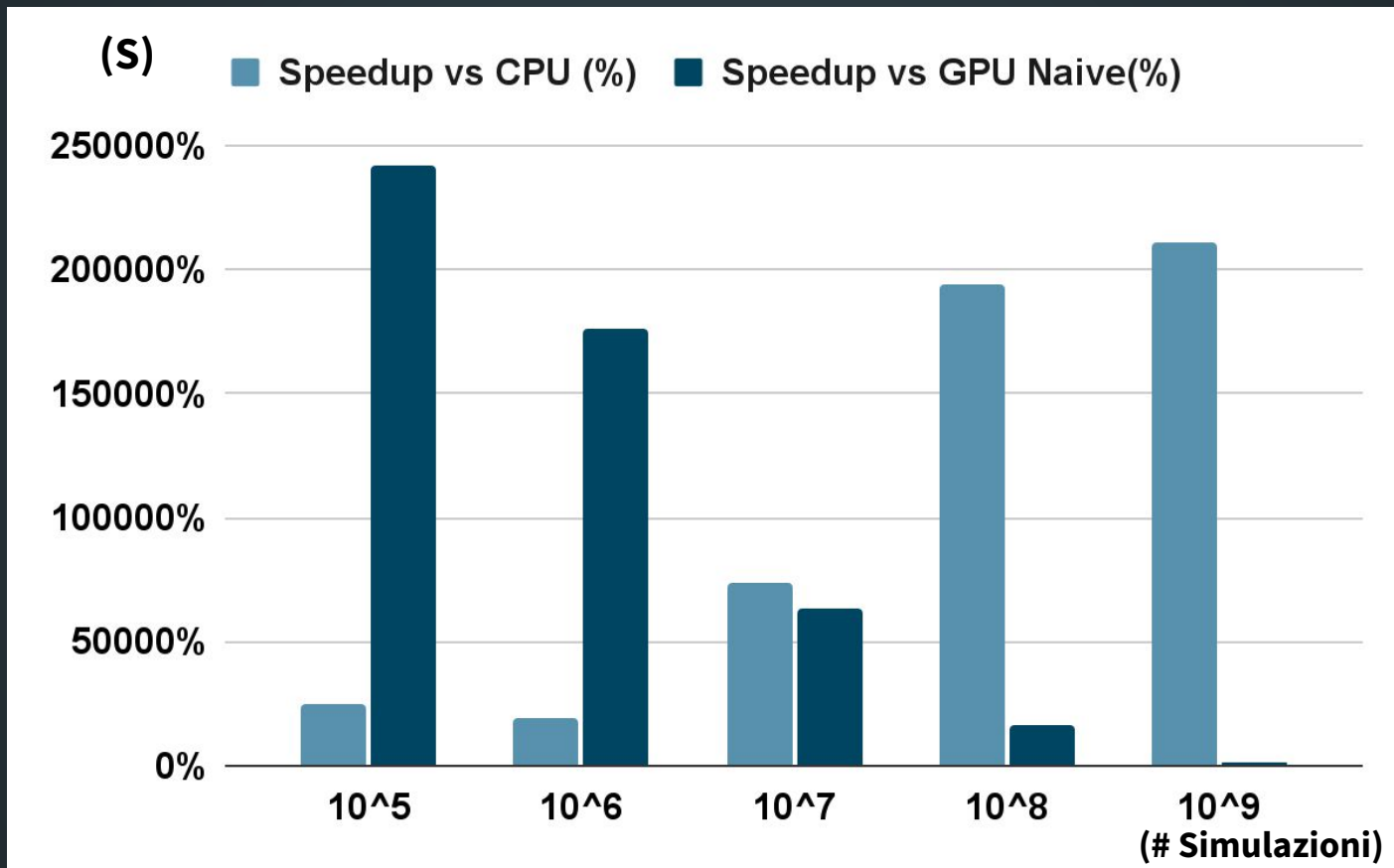


(# Simulazioni)

zoom

(# Simulazioni)

RISULTATI OTTENUTI E ANALISI: GPU OTTIMIZZATA



ALTRE STRATEGIE E ASPETTI DA CONSIDERARE

Per alcuni asset finanziari più particolari il calcolo del rendimento non è così banale. Questo può voler dire dover implementare simulazioni Monte Carlo Path Dependent

```
for (int i = 0; i < nSimulations; ++i) Naive CPU Path Dependent
```

```
{
```

```
    double currentPrice = S0;
```

```
    // Loop interno: cammina giorno per giorno
```

```
    for (int day = 0; day < DAYS_OPEN_IN_YEAR * T_YEARS; ++day)
```

```
    {
```

```
        double Z = distribution(generator);
```

```
        // Formula esponenziale incrementale
```

```
        currentPrice = currentPrice * std::exp(driftStep + volStep * Z);
```

```
    }
```

```
    // Salviamo il valore dell'indice raggiunto dalla simulazione
```

```
    simulatedPortfolioValues[i] = currentPrice;
```

```
}
```

Ogni nuovo valore
dipende dal precedente



GRAZIE MILLE PER L'ATTENZIONE



Maurizio Amadori, matricola 0001243119
Andrea Simonetti, matricola 0001221864